

American University of Armenia

2D and 3D Floorplanning: Simulated Annealing and Linear Programming Approaches

Capstone Project Report

Authors:

Zara Grigoryan

BS in Computer Science

zara_grigoryan@edu.aua.am

Anna Ishkhanyan

BS in Computer Science

anna_ishkhanyan@edu.aua.am

Supervisor:

Dr. Hayk Grigoryan

Yerevan, Armenia

May 2026

Abstract

Floorplanning is an early physical-design problem in which design decisions strongly influence chip area, wirelength, feasibility, and, in three-dimensional integrated circuits, vertical-interconnect and thermal behavior. This report presents a comparative implementation of sequence-based floorplanning methods for both 2D and 3D VLSI layouts. The framework combines simulated annealing topology search with two placement strategies: a construction-based decoder that rapidly generates compact placements from sequence representations, and a linear-programming-refined method that improves final coordinates and soft-block dimensions for a fixed topology.

The 2D implementation is based on sequence pairs and is inspired by the linear-programming floorplanning approach of Kim and Kim. The work extends this experimental framework to 3D floorplanning using sequence triples, from which horizontal, vertical, and inter-tier relations are decoded. The 3D formulation further incorporates tier assignment, TSV-related vertical-link cost, and lightweight thermal-aware penalties, allowing the comparison to capture constraints that arise in multi-tier layouts.

Experiments on MCNC benchmark circuits compare construction-only and LP-refined methods in both 2D and 3D using objective value, area, Half-Perimeter Wirelength, feasibility, TSV cost, thermal penalty, and runtime. The results show that LP refinement substantially improves final placement quality and feasibility relative to construction-only output, while the 3D sequence-triple extension achieves lower wirelength on larger benchmarks by exploiting inter-tier placement, at the cost of additional runtime and vertical-interconnect overhead. Overall, the project provides a modular implementation for studying the trade-offs between fast topology-driven construction, optimization-based refinement, and 3D-aware floorplanning.

Contents

Abstract	i
1 Introduction	1
2 Literature Review	3
2.1 Classical 2D Floorplanning and Representation-Based Methods	3
2.2 Linear Programming and Mathematical Programming Approaches	4
2.3 Simulated Annealing and Other Classical Metaheuristics	5
2.4 Transition from 2D to 3D Floorplanning	6
2.5 TSV-Aware and Thermal-Aware 3D Floorplanning	7
2.6 Multi-Objective Optimization in Floorplanning	8
2.7 Genetic, Swarm, Immune, and Hybrid Metaheuristics	9
2.8 Constraints Considered Across the Literature	9
2.9 Synthesis and Research Gap	10
3 Problem Formulation	13
3.1 Input Description	13
3.2 2D Floorplanning Problem	14
3.3 3D Floorplanning Problem	15
3.4 Objective Function and Constraints	16
3.5 Scope	17
4 Sequence-Based Floorplanning	18
4.1 Sequence Pair Representation	18
4.2 Sequence Triple Representation	19
4.3 Decoding and Precedence Constraints	20
4.4 DAG-Based Placement Construction	21
4.5 Role in This Project	22
5 Construction-Based Method	24
5.1 Method Overview	24
5.2 Simulated Annealing Search	25

5.3	Cost Evaluation	26
6	Linear Programming-Refined Method	27
6.1	Method Overview	27
6.2	LP Model	28
6.3	Coordinate and Dimension Refinement	29
6.4	Extension to 3D	30
7	Framework Implementation	31
7.1	System Architecture	31
7.2	Core Modules	31
7.3	Benchmark Parsing and Output Generation	32
7.4	Visualization and Solver Integration	33
8	Experimental Setup	34
8.1	Benchmarks	34
8.2	Compared Methods	34
8.3	Evaluation Metrics and Parameters	35
9	Results and Discussion	36
9.1	2D Results	36
9.2	3D Results	36
9.3	Construction-Based vs LP-Refined Comparison	37
9.4	Runtime and Trade-off Analysis	38
10	Conclusion and Future Work	40
10.1	Conclusion	40
10.2	Future Work	40
	Acknowledgment	42
	References	42

Chapter 1

Introduction

Integrated circuits are built from many functional components, such as memory blocks, logic units, arithmetic units, and control modules. Before the detailed placement of individual standard cells and before routing wires between them, the major blocks of the circuit must be arranged on the chip. This early physical design step is called floorplanning.

A floorplan can be understood as a high-level map of the chip. Each block is represented as a rectangle with a width and height, and the floorplanner must decide where each rectangle should be placed. The placement must satisfy basic geometric constraints: blocks on the same tier must not overlap, all blocks must fit inside the chip outline, and soft blocks must respect their allowed shape ranges. At the same time, connected blocks should be placed close to one another, because long interconnects increase wirelength, delay, power consumption, and routing difficulty.

Thus, floorplanning is both a geometric and combinatorial optimization problem. The geometry comes from assigning coordinates and dimensions to rectangular blocks. The combinatorial difficulty comes from choosing the relative ordering of many blocks. As the number of blocks increases, the number of possible arrangements grows rapidly. VLSI floorplanning is known to be NP-hard, and exact optimization is not practical for large designs [1].

Traditional floorplanning methods focus mainly on two-dimensional (2D) layouts, where all blocks are placed on one planar tier. In 2D floorplanning, the main goals are usually to reduce chip area, wirelength, dead space, and fixed-outline violations. However, as interconnect delay and density limits become increasingly important, three-dimensional integrated circuits (3D ICs) have become an attractive alternative. In a 3D IC, blocks can be distributed across multiple stacked tiers. This can reduce horizontal wirelength and chip footprint by allowing connected blocks to be placed above or below one another instead of far apart on a single plane.

The rapid growth of AI and LLM workloads is increasing demand for specialized accelerators with much higher compute density, memory bandwidth, and energy efficiency.

This is pushing chip design toward larger heterogeneous systems, chiplets, 2.5D/3D integration, and high-bandwidth memory, which in turn makes early physical-design decisions such as floorplanning increasingly important [17].

The 3D setting also introduces new constraints and trade-offs. Connections between different tiers require vertical interconnects, commonly modeled as Through-Silicon Vias (TSVs). TSVs are useful because they enable communication across tiers, but they also consume area and introduce routing and manufacturing costs. In addition, stacking active blocks can increase power density and worsen thermal behavior. Therefore, 3D floorplanning is not simply 2D floorplanning with an extra coordinate; it requires balancing wirelength reduction against TSV overhead, thermal effects, and tier-assignment decisions.

This report studies sequence-based floorplanning methods for both 2D and 3D layouts. In the 2D case, floorplans are represented using sequence pairs, which encode horizontal and vertical ordering relations between blocks using two permutations. In the 3D case, this representation is extended using sequence triples, where a third permutation is used to encode inter-tier ordering relations. These representations are useful because they separate the discrete topology of the floorplan from the continuous coordinate placement problem.

The project compares two placement strategies. The first is a construction-based method. Simulated annealing searches over sequence pairs in 2D and sequence triples in 3D. Each candidate sequence representation is decoded directly into block coordinates using precedence constraints and longest-path compaction. This method is fast enough to be used repeatedly during the annealing search.

The second strategy uses Linear Programming (LP) as a final refinement step. The 2D version is inspired by the LP-based sequence-pair floorplanning algorithm of Kim and Kim [3]. In that approach, simulated annealing searches over sequence-pair topologies, while an LP model optimizes coordinates and soft-block dimensions for a fixed ordering relation. This project implements the same principle for 2D floorplanning and extends it to 3D using sequence triples. In the 3D extension, simulated annealing first identifies a promising sequence-triple topology and tier assignment; then LP refinement improves the final in-tier coordinates and soft-block dimensions while preserving the decoded ordering constraints.

The goal of this report is to evaluate how representation choice and LP refinement affect floorplan quality and runtime. The experiments compare construction-only and LP-refined methods in both 2D and 3D. The methods are tested on MCNC benchmark circuits, including `apte`, `xerox`, `hp`, `ami33`, and `ami49`. The results are evaluated using objective value, footprint area, Half-Perimeter Wirelength (HPWL), TSV count, thermal-aware cost, feasibility, and runtime.

Chapter 2

Literature Review

VLSI floorplanning is one of the earliest and most influential stages of physical design. Given a set of circuit modules and their interconnections, the floorplanning task determines the relative placement of blocks before detailed placement and routing. Although the problem initially appears geometric, it is computationally difficult because the number of possible module arrangements, including rotations, mirroring or topological relationships (slicing and non-slicing representations) grows combinatorially with the number of blocks. Floorplanning is widely treated as an NP-hard combinatorial optimization problem. The main objectives in classical floorplanning are usually the minimization of chip area, dead space, and wirelength, while later work extends the problem to include fixed-outline constraints, thermal behavior, power, energy, leakage, TSV count, and performance-related constraints.

The literature on VLSI floorplanning can be organized into several major directions. Early and classical work focused on two-dimensional floorplanning representations and packing quality. Later studies introduced mathematical programming, especially linear programming, to obtain more accurate module coordinates from abstract representations. A large body of work then explored metaheuristic methods such as simulated annealing, genetic algorithms, particle swarm optimization, ant colony optimization, clonal selection, firefly optimization, and symbiosis-based search. With the emergence of three-dimensional integrated circuits, the field expanded toward 3D-specific concerns such as layer assignment, vertical interconnects, through-silicon vias (TSVs), thermal constraints, and multi-objective optimization.

2.1 Classical 2D Floorplanning and Representation-Based Methods

The foundation of floorplanning research lies in the representation of possible layouts. Since optimization algorithms do not usually operate directly on geometric coordinates,

they require compact encodings that can describe a large space of feasible module arrangements. Murata et al. [2] are central in this respect because their rectangle-packing-based module placement work introduced the sequence-pair representation, one of the most influential representations for nonslicing floorplans. A sequence pair encodes the horizontal and vertical ordering relations among modules using two permutations. From these two sequences, relative placement constraints can be derived, and the final coordinates can be computed by solving longest-path relations in horizontal and vertical constraint graphs. This representation is important because it can describe nonslicing floorplans more flexibly than slicing-tree or Polish-expression representations.

Singh, Baghel, and Agarwal [1] provide a broad review of VLSI floorplanning optimization using metaheuristic algorithms. Their work is useful as a general entry point because it summarizes the main floorplanning objectives, including area, wirelength, and dead space, and discusses how different representations affect the search space. They emphasize that the effectiveness of a metaheuristic is not determined only by the optimizer itself, but also by the representation used to encode floorplans. Common representations include sequence pairs, B*-trees, O-trees, Polish expressions, and transitive-closure graphs.

Tang [9] proposes a greedy algorithm for VLSI floorplan optimization using the O-tree representation. Unlike stochastic methods such as simulated annealing or genetic algorithms, this work focuses on a deterministic greedy strategy for exploring the floorplan search space. The floorplanning problem is treated as a permutation problem, and the objective is modeled as a weighted combination of area and interconnection cost, typically estimated using half-perimeter wirelength. This paper shows that floorplanning research has not been limited to complex metaheuristics; simpler constructive or greedy methods have also been studied as alternatives when runtime and implementation simplicity are important.

Together, these works show that 2D floorplanning research is strongly representation-driven. Representations define the structure of the search space and determine which constraints can be expressed efficiently. For this reason, representation choice remains a central issue even when the optimization method is changed.

2.2 Linear Programming and Mathematical Programming Approaches

While many floorplanning algorithms rely on heuristic decoding, Kim and Kim [3] propose a linear programming-based algorithm for floorplanning in VLSI design. Their method combines a sequence-pair representation with linear programming to determine module coordinates and dimensions. The sequence pair captures the relative ordering of modules,

while the LP model realizes a concrete floorplan by enforcing non-overlap, coordinate, and aspect-ratio constraints. This is particularly important for soft modules, whose widths and heights may vary within allowed bounds.

The contribution of Kim and Kim [3] lies in separating the floorplanning problem into two related levels: topology search and coordinate realization. Simulated annealing is used to explore different sequence pairs, while linear programming is used to compute high-quality layouts for a given topology. This hybrid structure suggests a principled way to connect combinatorial floorplan representations with optimization-based coordinate assignment. Instead of treating each candidate floorplan only through greedy packing, LP can be used to evaluate whether a given ordering admits a compact and feasible placement.

Additionally, this line of work makes the constraints of floorplanning explicit. Non-overlap constraints are derived from relative module ordering; aspect-ratio constraints restrict the shapes of soft blocks; and area and wirelength objectives are incorporated into the optimization.

2.3 Simulated Annealing and Other Classical Metaheuristics

Simulated annealing has been one of the most widely used methods in VLSI floorplanning because it is well suited for large combinatorial search spaces. Singh and Baghel [15] apply simulated annealing to reduce dead space in floorplans. Their method uses a B*-tree representation and standard perturbation operations such as moving, swapping, and rotating blocks. These moves generate neighboring floorplans, and simulated annealing accepts or rejects them according to a temperature-controlled probability rule. The objective includes area, interconnect cost, and dead space, making the method directly relevant to classical 2D floorplanning.

The main value of simulated annealing in floorplanning is its ability to escape local minima. Greedy methods can quickly become trapped in poor arrangements, while simulated annealing allows occasional uphill moves, especially in the early stages of the search. However, the drawback is runtime. High-quality simulated annealing often requires many iterations, careful temperature scheduling, and repeated floorplan decoding. This is one reason later research frequently combines simulated annealing with other methods rather than using it alone.

Abdullah et al. [8] apply a clonal selection algorithm to VLSI floorplanning. This method is inspired by artificial immune systems and uses selection, cloning, mutation, and replacement to evolve candidate floorplans. Their work uses an O-tree representation and focuses on producing optimal or near-optimal layouts for standard benchmarks. The

paper belongs to a broader category of evolutionary and bio-inspired algorithms that treat floorplanning as a search problem over encoded layouts.

Laudis et al. [4] introduce an adaptive symbiosis-based metaheuristic for combinatorial optimization in VLSI floorplanning. Their method modifies the Symbiotic Organisms Search algorithm and applies it to nonslicing floorplans represented by B*-trees. The algorithm adaptively selects among mutualism, commensalism, and parasitism phases depending on the quality of the population during the search. The objective is generally expressed as a weighted combination of area and wirelength, with additional attention to dead space and fixed-outline-related constraints. This paper represents a newer class of adaptive metaheuristics that attempt to improve search behavior by changing the optimization strategy during execution.

2.4 Transition from 2D to 3D Floorplanning

Three-dimensional integrated circuits introduce a major shift in floorplanning. In 2D floorplanning, all modules are placed on a single plane. In 3D floorplanning, modules may be distributed across multiple active layers, and communication between layers is achieved through vertical interconnects such as through-silicon vias (TSVs). This can reduce horizontal wirelength and improve integration density, but it also introduces new constraints: layer assignment, vertical overlap, TSV area, TSV count, thermal accumulation, and manufacturability.

Healy et al. [6] present a multiobjective microarchitectural floorplanning framework for both 2D and 3D ICs. Their work moves floorplanning beyond abstract benchmark blocks and studies processor microarchitectural units. The objectives include not only area and wirelength but also performance and thermal reliability. In 3D ICs, placing strongly communicating blocks on adjacent tiers can reduce communication delay, but clustering active blocks may increase temperature. Thus, the paper highlights a fundamental conflict in 3D floorplanning: the floorplan that is best for performance may not be best for thermal behavior.

Healy et al. [6] also discuss 3D-specific issues such as vertical overlap optimization and bonding-aware layer partitioning. Their method combines linear programming with simulated annealing, making it relevant both to mathematical programming and metaheuristic categories. The significance of this paper is that it frames 3D floorplanning as a system-level multi-objective problem rather than a simple geometric extension of 2D packing.

Lu et al. [5] focus specifically on TSV-aware floorplanning for 3D integrated circuits. Their work considers the trade-off between critical wirelength reduction and TSV cost. Since moving modules across tiers may reduce long critical interconnects but increase the number of required TSVs, the algorithm must respect constraints on both maximum

critical length and maximum TSV count. This paper treats 3D floorplanning as a constrained optimization problem, where performance improvement is limited by physical and manufacturing costs associated with vertical interconnects.

2.5 TSV-Aware and Thermal-Aware 3D Floorplanning

TSVs are one of the defining features of 3D IC floorplanning. Early 3D floorplanning methods sometimes treated TSVs abstractly, either as cost terms or dimensionless vertical connections. Later work showed that this is unrealistic because TSVs occupy physical area, require keep-out regions, and affect routability.

Tsai, Wang, and Hwang [12] address through-silicon via planning in 3D floorplanning, challenging the simplification of treating TSVs as ideal points. They show that ignoring TSV area and placement can underestimate wirelength and produce infeasible designs. Their two-stage fixed-outline algorithm first plans macros together with TSV blocks and then refines the result through TSV reassignment. This makes the paper a key reference for TSV-aware 3D floorplanning because it integrates TSV feasibility into the floorplanning process rather than leaving it as post-processing.

Li, Mak, and Wang [13] further develop TSV-aware fixed-outline 3D floorplanning by proposing a fast algorithm with TSV co-placement. The motivation is that TSV-unaware floorplans often require expensive whitespace redistribution later. By placing functional modules and TSVs together, the algorithm improves the realism of wirelength estimation and routability. This paper especially accounts for scalability, as TSV-aware methods can become computationally expensive if TSV placement is handled too late or too separately from macro placement.

Li et al. [13] focus on thermal via planning and its application in 3D floorplanning. Unlike signal TSVs, thermal vias are inserted primarily to improve vertical heat conduction. The paper formulates parts of the thermal via planning problem using convex programming ideas and integrates thermal via planning into a two-stage 3D floorplanning flow. This contribution separates two different TSV-related concerns: signal TSVs support communication, while thermal vias support heat removal. Both occupy area and affect routing, but their optimization goals differ.

Thermal constraints are another major issue in 3D ICs. Since active layers are stacked vertically, heat dissipation becomes more difficult than in 2D designs. Huang et al. [10] propose a thermal-driven force-directed floorplanning algorithm for 3D ICs. Force-directed floorplanning methods model modules as physical objects and translate design objectives into artificial forces. Interconnected modules may attract one another to reduce wirelength, while overlap, congestion, or thermal interactions may introduce

repulsive forces that separate modules and guide the layout toward a feasible solution. Force-directed methods are attractive because they can provide more scalable alternatives to simulated annealing. Instead of exploring floorplans through random perturbations, force-directed placement models modules and objectives through physical analogies, gradually moving blocks toward lower-cost configurations. In the 3D setting, this allows thermal effects to influence placement decisions directly.

Xiao et al. [11] address fixed-outline thermal-aware 3D floorplanning. Fixed-outline constraints are important because real chips often have predetermined die dimensions. A floorplan that minimizes area in theory may be unusable if it violates a required outline. Their work combines thermal modeling, whitespace redistribution, and thermal TSV insertion. The key idea is that whitespace is not merely unused area; in 3D floorplanning, it can be strategically aligned across tiers to support thermal via insertion and reduce hotspots.

Together, these TSV-aware and thermal-aware papers show that 3D floorplanning must be physically informed. It is not enough to assign modules to layers and estimate 3D wirelength. A realistic 3D floorplanner must account for TSV count, TSV area, TSV position, thermal via insertion, whitespace alignment, and peak temperature.

2.6 Multi-Objective Optimization in Floorplanning

Modern floorplanning is inherently multi-objective. Even in 2D, area and wirelength often conflict. In 3D, the conflict becomes stronger because thermal behavior, performance, energy, TSV count, leakage, and fixed-outline feasibility must also be considered.

Healy et al. [6] are again relevant in this category because their microarchitectural floorplanner explicitly studies trade-offs among performance, area, wirelength, and thermal behavior. This is a more meaningful form of multi-objective optimization than simply combining area and wirelength into one weighted cost. Their work shows that floorplanning decisions affect architectural performance and reliability, especially in 3D processors.

Srinivasan and Venkatesan [7] propose an enhanced firefly optimization method for energy- and heat-aware VLSI floorplanning. The method uses a non-slicing B*-tree representation and evaluates candidate floorplans using objectives related to heat generation, space usage, and wirelength. This paper reflects a trend in which floorplanning objectives are extended beyond geometric compactness. Heat and energy are included directly in the optimization function, making the method relevant to power- and thermal-aware physical design.

The same broader trend appears in later hybrid metaheuristic work. Although not all such papers introduce fundamentally new floorplan models, they are useful because they show how the objective function of floorplanning has expanded. Area and wire-

length are still central, but they are increasingly combined with heat, energy, TSV count, performance, and outline constraints.

2.7 Genetic, Swarm, Immune, and Hybrid Metaheuristics

A large part of the floorplanning literature applies bio-inspired and nature-inspired algorithms. Genetic algorithms, simulated annealing, ant colony optimization, particle swarm optimization, clonal selection, firefly optimization, and symbiosis-based search have all been used because they are flexible and can handle nonlinear, discrete, and multi-objective search spaces.

Singh, Baghel, and Agarwal [1] survey many of these methods and compare their use in VLSI floorplanning. The paper emphasizes that performance depends not only on the metaheuristic but also on representation, benchmark choice, cost-function design, and constraint handling.

Abdullah et al. [8] represent the immune-inspired branch through their clonal selection algorithm. Tang [9] represents a greedy, representation-driven alternative. Singh and Baghel [15] represent the simulated-annealing branch, especially for dead-space reduction with B*-trees. Laudis et al. [4] represent adaptive symbiosis-based optimization, while Srinivasan and Venkatesan [7] represent swarm-based multi-objective optimization through enhanced firefly search.

These methods differ in their search strategies. Simulated annealing uses probabilistic acceptance to escape local minima. Genetic and immune-inspired algorithms evolve populations of candidate floorplans. Swarm-based algorithms model collective movement toward better solutions. Symbiosis-based methods model interactions between organisms and adaptively select search phases. However, many of these papers share similar benchmark settings and often optimize variations of area, wirelength, and dead space. Their real difference lies in convergence behavior, robustness, representation compatibility, and ability to handle additional objectives.

These methods are evidence that VLSI floorplanning has long been treated as a benchmark problem for combinatorial optimization. A careful review should therefore avoid presenting every metaheuristic as equally novel. Instead, it should ask what each method contributes: a better representation, a stronger search process, a new objective, improved runtime, or more realistic physical constraints.

2.8 Constraints Considered Across the Literature

Across the reviewed papers, the constraints and objectives can be grouped as follows.

Classical 2D floorplanning papers mainly consider non-overlap, area minimization, dead-space reduction, aspect-ratio constraints for soft modules, and wirelength minimization. Murata et al. [2], Kim and Kim [3], Tang [9], Singh and Baghel [15], and Laudis et al. [4] belong mainly to this category.

Metaheuristic papers usually add algorithmic flexibility but often retain similar physical constraints. Singh et al. [1], Abdullah et al. [8], Singh and Baghel [15], Laudis et al. [4], and Srinivasan and Venkatesan [7] optimize combinations of area, wirelength, dead space, heat, and energy depending on the specific method.

3D floorplanning papers introduce layer assignment, vertical interconnect cost, TSV count, TSV placement, thermal constraints, and sometimes fixed-outline feasibility. Lu et al. [5], Healy et al. [6], Huang et al. [10], Xiao et al. [11], Tsai et al. [12], Li et al. [13], and Li et al. [14] are central in this category.

The most physically realistic 3D works are those that do not treat the third dimension merely as an added coordinate. Tsai et al. [12] and Li et al. [13] explicitly model TSV placement, while Xiao et al. [11] and Li et al. [14] incorporate thermal via planning and whitespace considerations. Healy et al. [6] further expands the objective space by incorporating microarchitectural performance and thermal reliability.

2.9 Synthesis and Research Gap

The reviewed literature shows a clear progression. Early floorplanning research focused on 2D geometric packing, where the main challenges were representation, area minimization, dead-space reduction, and wirelength estimation. Sequence pairs, O-trees, and B*-trees became important because they allowed algorithms to search large spaces of slicing and nonslicing floorplans efficiently. Mathematical programming methods, especially the LP-based method of Kim and Kim [3], introduced a more explicit way to realize coordinates and dimensions under non-overlap and aspect-ratio constraints.

The next major direction was the use of metaheuristics. Simulated annealing, clonal selection, firefly optimization, symbiosis-based search, and other population-based methods were applied to improve solution quality in a highly non-convex search space. These methods are valuable because floorplanning is NP-hard and exact optimization is not scalable for large instances. However, many metaheuristic papers mainly change the search method while preserving similar objective functions.

The emergence of 3D ICs significantly expanded the problem. In 3D floorplanning, the optimizer must consider not only area and wirelength but also layer assignment, vertical interconnects, TSV count, TSV area, thermal behavior, and fixed-outline feasibility. TSV-aware works show that vertical interconnects cannot be modeled only as abstract penalties; they are physical objects that consume area and affect routing. Thermal-aware works show that floorplanning must account for heat early because poor thermal decisions

cannot always be repaired in later design stages.

For this project, the literature suggests a natural research direction: begin with classical 2D floorplanning representations and algorithms, evaluate them on standard benchmarks, and then formulate extensions toward 3D by adding layer assignment, TSV-related cost, and possibly thermal or fixed-outline constraints. The most relevant methodological bridge is the combination of representation-based search with mathematical optimization. In particular, sequence-pair or B*-tree representations can define relative placement topologies, while LP-based models can be used to compute coordinates under explicit constraints. A further extension would be to compare constructive, LP-based, and simulated-annealing-based realizations of the same benchmark instances, then study how the formulation changes when modules are assigned to multiple tiers.

Overall, the literature shows that floorplanning has evolved from a 2D packing problem into a multi-objective, physically constrained optimization problem. The remaining challenge is not merely to apply another metaheuristic, but to design a formulation that clearly connects representation, constraints, objective functions, and realistic 3D design requirements.

Table 2.1: Summary of major objectives and constraints in reviewed floorplanning literature

Category	Representative Works	Main Objectives / Constraints
Classical 2D representation-based floorplanning	Murata et al. [2], Tang [9], Singh et al. [1]	Area, wirelength, dead space, non-overlap, compact representation, efficient decoding
LP-based floorplanning	Kim and Kim [3], Healy et al. [6]	Coordinate realization, non-overlap constraints, aspect-ratio constraints, area and wirelength optimization, topology-to-layout conversion
Simulated annealing and classical metaheuristics	Singh and Baghel [15], Singh et al. [1]	Area, wirelength, dead-space reduction, stochastic neighborhood search, block rotation, node movement, node swapping
Bio-inspired and swarm-based metaheuristics	Laudis et al. [4], Abdullah et al. [8], Srinivasan and Venkatesan [7]	Area, wirelength, dead space, heat, energy, population-based search, adaptive search behavior
3D floorplanning	Lu et al. [5], Healy et al. [6]	Layer assignment, vertical interconnects, performance, thermal reliability, TSV count, critical wirelength
TSV-aware 3D floorplanning	Tsai et al. [12], Li et al. [13]	TSV area, TSV placement, TSV count, routability, fixed-outline feasibility, wirelength estimation
Thermal-aware 3D floorplanning	Huang et al. [10], Xiao et al. [11], Li et al. [14]	Peak temperature, thermal vias, whitespace redistribution, heat conduction, thermal feasibility
Multi-objective floorplanning	Healy et al. [6], Srinivasan and Venkatesan [7]	Area, wirelength, performance, energy, heat, thermal reliability, system-level trade-offs

Chapter 3

Problem Formulation

The floorplanning problem considered in this project is formulated as a constrained optimization problem. Given a set of rectangular circuit blocks and a set of nets connecting them, the goal is to determine block positions, dimensions, and, in the 3D case, layer assignments, such that blocks do not overlap and the resulting layout minimizes a weighted objective involving chip size, wirelength, and 3D-specific costs.

The problem contains both discrete and continuous decisions. The relative ordering of blocks, block rotations, and layer assignments are discrete decisions, while block coordinates and soft-block dimensions are continuous decisions. Solving the complete problem exactly would require a mixed-integer or nonlinear formulation. Therefore, this project separates the problem into two stages: a sequence-based topology search using simulated annealing, and a coordinate refinement stage using construction or linear programming.

3.1 Input Description

The input consists of a set of circuit blocks and a set of nets. Let

$$B = \{1, 2, \dots, n\}$$

denote the set of blocks, and let

$$N = \{1, 2, \dots, m\}$$

denote the set of nets.

A block represents a rectangular circuit module, such as a memory unit, arithmetic unit, or other functional component. A net represents an electrical connection between two or more blocks. In a circuit, blocks are not placed independently: connected blocks should generally be placed close to one another to reduce interconnect length, delay, and routing complexity. Therefore, nets are used to estimate the communication cost of a

floorplan.

Formally, each net $k \in N$ connects a subset of blocks:

$$N_k \subseteq B.$$

Each block $i \in B$ is represented as a rectangle. Blocks may be either hard or soft. A hard block has fixed dimensions:

$$w_i = \bar{w}_i, \quad h_i = \bar{h}_i,$$

up to rotation. If rotation is allowed, the width and height may be swapped.

A soft block has fixed area A_i but variable width and height. Its dimensions must satisfy aspect-ratio constraints:

$$\begin{aligned} A_i &\leq w_i h_i, \\ r_i^{\min} &\leq \frac{h_i}{w_i} \leq r_i^{\max}, \end{aligned}$$

where $r_i^{\min}$ and $r_i^{\max}$ are the minimum and maximum allowed aspect ratios.

Wirelength is estimated using half-perimeter wirelength (HPWL), computed from the centers of the blocks connected by each net.

The input may also specify a fixed outline. If a fixed outline is used, the final floorplan must fit within given chip dimensions:

$$W = W_0, \quad H = H_0.$$

Otherwise, W and H are optimization variables representing the width and height of the resulting chip outline.

3.2 2D Floorplanning Problem

In the 2D floorplanning problem, all blocks are placed on a single layer. Each block i has lower-left coordinate (x_i, y_i) and dimensions (w_i, h_i) . The block must lie inside the chip boundary:

$$\begin{aligned} x_i &\geq 0, & y_i &\geq 0, \\ x_i + w_i &\leq W, \\ y_i + h_i &\leq H. \end{aligned}$$

For any two distinct blocks i and j , non-overlap requires that at least one of the following relations holds:

$$x_i + w_i \leq x_j,$$

$$x_j + w_j \leq x_i,$$

$$y_i + h_i \leq y_j,$$

$$y_j + h_j \leq y_i.$$

Directly modeling this disjunction requires binary variables. Instead, this project uses a sequence-pair representation to determine one relative relation for each pair of blocks. Once the relation is fixed, the corresponding non-overlap constraint becomes linear.

For example, if the sequence pair implies that block i is to the left of block j , then the following linear constraint is added:

$$x_i + w_i \leq x_j.$$

If block i is below block j , the corresponding constraint is:

$$y_i + h_i \leq y_j.$$

Thus, the sequence pair removes the need to explicitly choose among all non-overlap disjunctions during coordinate optimization.

3.3 3D Floorplanning Problem

In the 3D floorplanning problem, the design is distributed across multiple stacked tiers. Each tier contains a two-dimensional placement region, and blocks may be assigned to different tiers. Let L denote the number of available tiers. Each block i is assigned to one tier:

$$\ell_i \in \{0, 1, \dots, L - 1\}.$$

Blocks assigned to the same tier must not overlap in the x - y plane. Therefore, for two blocks i and j such that $\ell_i = \ell_j$, one horizontal or vertical separation constraint must hold.

If two blocks are assigned to different tiers, they are allowed to overlap in the x - y plane because they occupy different physical levels of the 3D stack. However, nets connecting blocks on different tiers require vertical interconnects, such as through-silicon vias (TSVs). These vertical interconnects introduce additional area, routing, and manufacturing costs, so they are included in the 3D objective.

The 3D extension therefore adds two main decisions to the 2D problem:

tier assignment: ℓ_i ,

vertical interconnect cost between connected blocks on different tiers.

In this project, sequence triples are used to encode both in-tier ordering relations and inter-tier relations. For blocks on the same tier, horizontal and vertical precedence constraints are generated similarly to the 2D case. For blocks assigned to different tiers, no in-plane non-overlap constraint is needed.

3.4 Objective Function and Constraints

The general objective combines chip size, wirelength, and 3D-specific costs. For 2D floorplanning, the objective is written as:

$$\min \quad \alpha(W + H) + \beta \sum_{k \in N} \text{HPWL}_k,$$

where α controls the importance of chip size and β controls the importance of wirelength.

The HPWL of net k is computed from the centers of its connected blocks. The center of block i is:

$$c_i^x = x_i + \frac{1}{2}w_i, \quad c_i^y = y_i + \frac{1}{2}h_i.$$

Then:

$$\text{HPWL}_k = \left(\max_{i \in N_k} c_i^x - \min_{i \in N_k} c_i^x \right) + \left(\max_{i \in N_k} c_i^y - \min_{i \in N_k} c_i^y \right).$$

For 3D floorplanning, the objective is extended as:

$$\min \quad \alpha(W + H) + \beta \sum_{k \in N} \text{HPWL}_k + \gamma \cdot \text{TSV} + \delta \cdot \text{ThermalCost} + \eta \cdot \text{KeepoutCost}.$$

Here, TSV represents the vertical interconnect cost caused by nets spanning multiple layers, ThermalCost is a lightweight thermal-aware penalty, and KeepoutCost penalizes TSV-related spacing conflicts.

In fixed-outline experiments, W and H are fixed constants:

$$W = W_0, \quad H = H_0.$$

In this case, the chip-size term does not influence optimization directly, and the main optimization pressure comes from wirelength, feasibility, TSV count, and thermal-related costs.

The main constraints are summarized as follows:

$$x_i \geq 0, \quad y_i \geq 0,$$

$$x_i + w_i \leq W,$$

$$y_i + h_i \leq H,$$

non-overlap constraints for all block pairs on the same layer,

$$r_i^{\min} w_i \leq h_i \leq r_i^{\max} w_i \quad \text{for soft blocks,}$$

$$w_i h_i \geq A_i \quad \text{for soft blocks.}$$

The exact soft-block area constraint $w_i h_i \geq A_i$ is nonlinear. In the LP-refined method, it is approximated using linear surrogate constraints and corrected iteratively. This follows the idea that the LP should remain linear while still producing dimensions that satisfy the required block areas.

Overall, the formulation separates the difficult discrete topology decisions from the continuous coordinate optimization. Sequence pairs and sequence triples define the relative placement topology, construction methods provide fast coordinate generation, and LP refinement improves the final placement for a fixed topology.

3.5 Scope

This work focuses on early block-level floorplanning objectives, including chip size, wirelength, feasibility, TSV-related vertical communication cost, and lightweight thermal penalties. It does not explicitly model clock-tree construction, clock skew, or clock insertion delay. In a conventional physical-design flow, detailed clock distribution is handled during clock tree synthesis after placement information is available, while the present framework operates at an earlier floorplanning stage [18]. Incorporating clock-aware floorplanning or timing-driven clock-distribution proxies is therefore left as future work.

Chapter 4

Sequence-Based Floorplanning

Floorplanning algorithms usually do not search directly over block coordinates. Direct coordinate search is difficult because every candidate placement must satisfy non-overlap constraints, boundary constraints, and, for soft blocks, dimension constraints. Instead, many floorplanning methods use a compact representation that encodes the relative topology of the blocks. Coordinates are then generated from this representation.

This project uses sequence-based representations because they provide a natural link between combinatorial topology search and coordinate optimization. In 2D, sequence pairs encode the relative horizontal and vertical ordering of blocks. In 3D, sequence triples extend this idea by also encoding inter-layer ordering. These representations are well suited to simulated annealing because small mutations of the sequences produce neighboring floorplan topologies. They are also well suited to LP refinement because, once the ordering relations are fixed, non-overlap constraints become linear inequalities.

The main reasons for choosing sequence-based floorplanning are therefore:

1. Sequence pairs can represent general non-slicing floorplans, not only slicing layouts.
2. The representation is compact, using permutations of block indices.
3. Relative placement constraints can be decoded systematically.
4. The decoded constraints can be used both for construction-based placement and for LP-based coordinate refinement.
5. The representation extends naturally to 3D through sequence triples.

4.1 Sequence Pair Representation

A sequence pair is a topology representation for two-dimensional rectangular floorplanning. It was introduced by Murata et al. [2] and has become one of the standard representations for non-slicing floorplans. For a circuit with n blocks, a sequence pair consists

of two permutations of the block indices:

$$\Gamma = (\Gamma^+, \Gamma^-),$$

where

$$\Gamma^+ = (\gamma_1^+, \gamma_2^+, \dots, \gamma_n^+), \quad \Gamma^- = (\gamma_1^-, \gamma_2^-, \dots, \gamma_n^-).$$

Let $\text{pos}^+(i)$ and $\text{pos}^-(i)$ denote the positions of block i in Γ^+ and Γ^- , respectively. For any pair of blocks i and j , their relative position is determined by comparing their order in the two sequences.

If i appears before j in both sequences, then i is placed to the left of j :

$$\text{pos}^+(i) < \text{pos}^+(j), \quad \text{pos}^-(i) < \text{pos}^-(j) \implies i \text{ is left of } j.$$

If i appears after j in both sequences, then i is placed to the right of j :

$$\text{pos}^+(i) > \text{pos}^+(j), \quad \text{pos}^-(i) > \text{pos}^-(j) \implies i \text{ is right of } j.$$

If i appears after j in Γ^+ but before j in Γ^- , then i is placed below j :

$$\text{pos}^+(i) > \text{pos}^+(j), \quad \text{pos}^-(i) < \text{pos}^-(j) \implies i \text{ is below } j.$$

If i appears before j in Γ^+ but after j in Γ^- , then i is placed above j :

$$\text{pos}^+(i) < \text{pos}^+(j), \quad \text{pos}^-(i) > \text{pos}^-(j) \implies i \text{ is above } j.$$

Thus, the sequence pair encodes one relative placement relation for every pair of blocks. This avoids explicitly storing coordinates during topology search.

4.2 Sequence Triple Representation

For 3D floorplanning, this project extends the sequence-pair idea to a sequence-triple representation. A sequence triple consists of three permutations:

$$\Theta = (\Gamma^1, \Gamma^2, \Gamma^3).$$

The first two sequences preserve the idea of horizontal and vertical ordering, while the third sequence introduces layer-ordering information. For two blocks i and j , assume without loss of generality that i appears before j in Γ^1 . Their relation is then decoded using their relative order in Γ^2 and Γ^3 .

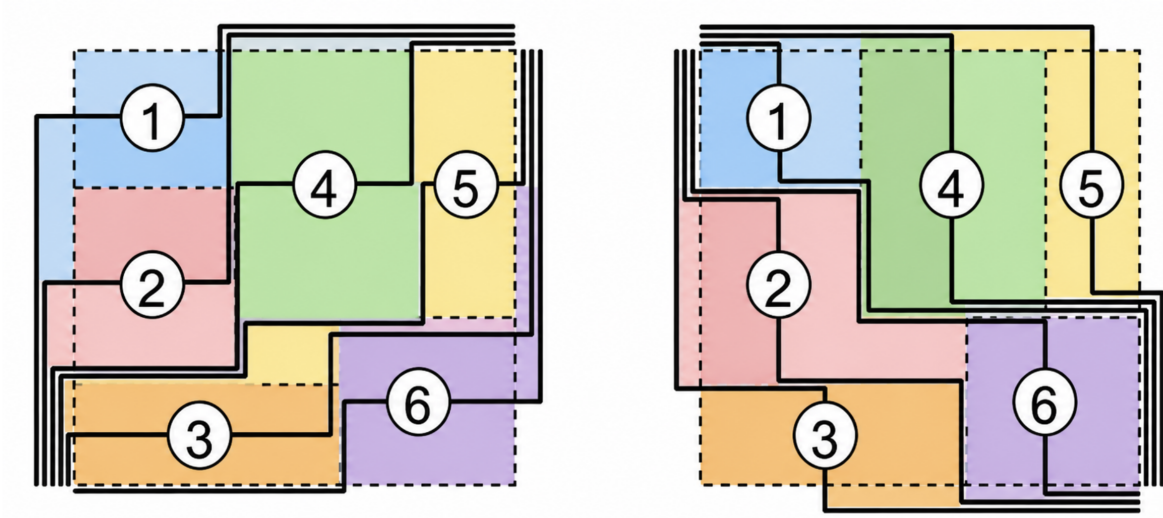


Figure 4.1: Illustration of the sequence-pair representation for a non-slicing floorplan. The two sequences encode the relative ordering of the blocks, from which horizontal and vertical placement relations can be derived.

If i also appears before j in Γ^2 , then i is placed to the left of j :

$$\text{pos}^1(i) < \text{pos}^1(j), \quad \text{pos}^2(i) < \text{pos}^2(j) \implies i \text{ is left of } j.$$

If i appears after j in Γ^2 but before j in Γ^3 , then i is placed below j :

$$\text{pos}^1(i) < \text{pos}^1(j), \quad \text{pos}^2(i) > \text{pos}^2(j), \quad \text{pos}^3(i) < \text{pos}^3(j) \implies i \text{ is below } j.$$

Otherwise, the pair is interpreted as an inter-layer relation:

$$\text{pos}^1(i) < \text{pos}^1(j), \quad \text{pos}^2(i) > \text{pos}^2(j), \quad \text{pos}^3(i) > \text{pos}^3(j) \implies i \text{ is on a lower layer than } j.$$

This representation allows the optimizer to decide whether two blocks should be separated horizontally, separated vertically, or assigned to different layers. Blocks on different layers may overlap in the x - y plane, while blocks on the same layer must satisfy ordinary two-dimensional non-overlap constraints.

4.3 Decoding and Precedence Constraints

Decoding converts a sequence representation into pairwise geometric relations. In 2D, each pair of blocks is decoded into one of four relations:

$$\text{LEFT_OF}, \quad \text{RIGHT_OF}, \quad \text{BELOW}, \quad \text{ABOVE}.$$

In 3D, the sequence triple adds an inter-layer relation:

LOWER_LAYER.

After decoding, each relation is converted into a precedence constraint. For example, if block i is left of block j , then:

$$x_i + w_i \leq x_j.$$

If block i is below block j , then:

$$y_i + h_i \leq y_j.$$

These constraints guarantee non-overlap. In 3D, these constraints are only required for blocks assigned to the same layer.

4.4 DAG-Based Placement Construction

For construction-based placement, the decoded precedence constraints are represented as directed acyclic graphs. In 2D, two graphs are built:

G_x = horizontal constraint graph,

G_y = vertical constraint graph.

If block i is left of block j , an edge is added to the horizontal graph:

$$i \rightarrow j$$

with weight w_i . This edge represents:

$$x_j \geq x_i + w_i.$$

If block i is below block j , an edge is added to the vertical graph:

$$i \rightarrow j$$

with weight h_i . This edge represents:

$$y_j \geq y_i + h_i.$$

The final coordinates are obtained by computing longest paths in these graphs. The horizontal longest-path distances give the x -coordinates, and the vertical longest-path

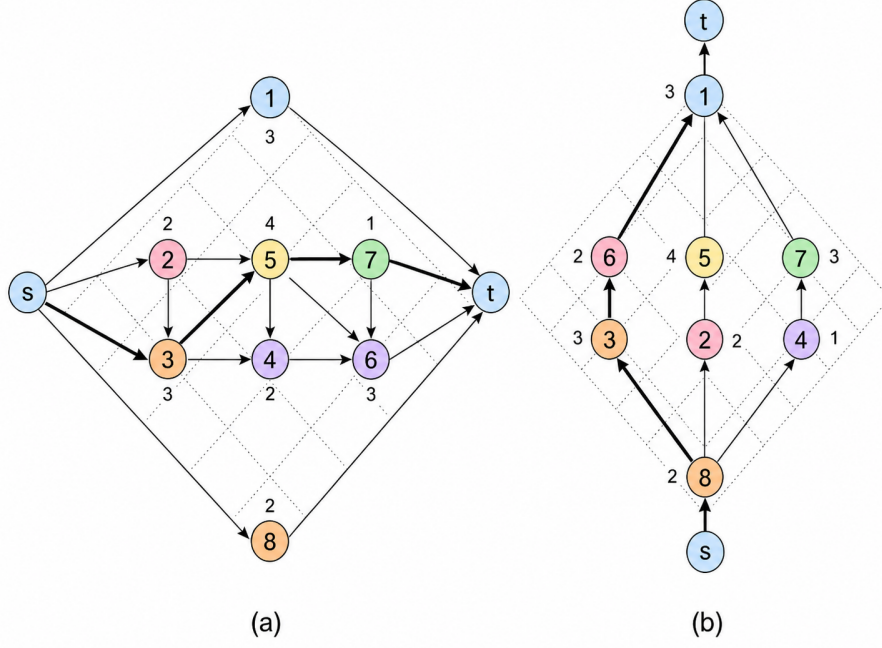


Figure 4.2: DAG-based construction of a floorplan from precedence constraints. Horizontal and vertical ordering relations are encoded as directed acyclic graphs, and longest-path distances in these graphs determine the final block coordinates.

distances give the y -coordinates:

$$x_i = d_x(i), \quad y_i = d_y(i).$$

The resulting placement is compact because each block is placed as early as possible while satisfying all precedence constraints. In 3D, the same idea is applied layer by layer. The sequence triple is decoded into horizontal, vertical, and inter-tier relations, from which the tier assignment used by the construction and LP stages is derived. Then, for each layer, horizontal and vertical constraint graphs are constructed for the blocks on that layer. Longest-path computation is then used to obtain compact coordinates within each layer.

4.5 Role in This Project

Sequence-based representations are used throughout the project as the main topology encoding. In the construction-based method, simulated annealing mutates sequence pairs or sequence triples, and each candidate is decoded into a placement using the DAG-based construction method.

In the LP-refined method, simulated annealing still searches over sequence representations. However, after the best topology is found, the decoded pairwise relations are passed to a linear programming model. The LP then refines coordinates and soft-block

dimensions while preserving the topology implied by the sequence pair or sequence triple.

In this way, the sequence representation serves as the common link between topology search and final placement generation in both methods.

Chapter 5

Construction-Based Method

The construction-based method is the fast placement procedure used to evaluate sequence-pair and sequence-triple candidates during simulated annealing. Instead of solving an optimization model for every candidate topology, the method directly decodes the sequence representation and constructs a compact placement using precedence constraints. This makes it suitable for repeated evaluation inside a stochastic search loop.

5.1 Method Overview

For a fixed sequence representation, the construction method performs three main steps. First, it determines the relative ordering constraints implied by the sequence pair or sequence triple. Second, it assigns block dimensions. Third, it computes a compact placement from the resulting constraints.

For hard blocks, the method considers the two possible rectangular orientations:

$$(w_i, h_i) = (\bar{w}_i, \bar{h}_i)$$

and

$$(w_i, h_i) = (\bar{h}_i, \bar{w}_i).$$

For soft blocks, the method considers a finite set of candidate aspect ratios between the minimum and maximum allowed values. In the implementation, five logarithmically spaced aspect-ratio candidates are used by default. For a soft block with area A_i and candidate aspect ratio r , the corresponding dimensions are:

$$w_i = \sqrt{\frac{A_i}{r}}, \quad h_i = \sqrt{A_i r}.$$

The candidates are selected in a block-effect order, so larger or more shape-constrained blocks are fixed earlier. For each block, the method tries its candidate dimensions, con-

structs the corresponding compact placement, evaluates the objective, and keeps the best candidate. This follows the structure of the construction method used by Kim and Kim [3], while using a small discrete set of soft-block aspect ratios for practical experimentation.

Once dimensions are assigned, coordinates are computed from the decoded precedence constraints. As described in the previous chapter, left-right relations define horizontal constraints and below-above relations define vertical constraints. The placement is then obtained by longest-path computation in the corresponding directed acyclic graphs. In the 3D case, the same procedure is applied within each tier after the sequence triple determines tier relations.

5.2 Simulated Annealing Search

The construction method is used as the evaluation function inside simulated annealing. Simulated annealing searches over the discrete topology space by repeatedly perturbing the current sequence representation. In 2D, the state is a sequence pair. In 3D, the state is a sequence triple.

The neighborhood moves used in the implementation are simple permutation operations. They include swapping two blocks in both sequences, swapping two blocks in one sequence, and moving one block before another in one sequence. In the 3D case, the same types of operations are applied to the three sequences of the sequence triple. These moves change the relative ordering relations and therefore generate a neighboring floorplan topology.

At each iteration, a candidate sequence is generated and evaluated by the construction method. If the candidate has lower cost, it is accepted. If it has higher cost, it may still be accepted with probability

$$P = \exp\left(-\frac{\Delta C}{T}\right),$$

where ΔC is the increase in cost and T is the current temperature. The temperature is gradually reduced using a cooling factor. This allows the search to accept worse moves early in the run, helping it escape local minima, while becoming more selective as the temperature decreases.

The role of simulated annealing is therefore not to compute coordinates directly. Instead, it searches over sequence representations. The construction method then converts each candidate representation into a concrete placement and objective value.

5.3 Cost Evaluation

The cost function measures the quality of the constructed placement. For 2D floorplanning, the primary terms are chip size and wirelength. The wirelength is estimated using HPWL over net bounding boxes:

$$\text{HPWL}_k = \left(\max_{i \in N_k} c_i^x - \min_{i \in N_k} c_i^x \right) + \left(\max_{i \in N_k} c_i^y - \min_{i \in N_k} c_i^y \right).$$

The 2D construction objective is written as:

$$C_{2D} = \alpha(W + H) + \beta \sum_{k \in N} \text{HPWL}_k + P_{\text{outline}},$$

where P_{outline} is a penalty applied when the constructed placement violates fixed-outline or aspect-ratio requirements.

For 3D floorplanning, the cost function includes additional terms:

$$C_{3D} = \alpha(W + H) + \beta \sum_{k \in N} \text{HPWL}_k + \gamma \cdot \text{TSV} + \delta \cdot \text{ThermalCost} + \eta \cdot \text{KeepoutCost} + P_{\text{outline}}.$$

The TSV term penalizes nets that connect blocks placed on different tiers. The thermal term is a lightweight hotspot proxy used to discourage highly concentrated activity in the 3D stack. The keepout term represents additional cost associated with TSV spacing. These terms allow the construction method to compare 3D placements using more than only horizontal wirelength.

The construction-based method is approximate, but it is computationally efficient. This efficiency is important because simulated annealing evaluates many candidate sequence representations. A full LP solve at every iteration would be much more expensive, so construction is used during the search and LP refinement is applied only after a strong candidate topology has been found.

Chapter 6

Linear Programming-Refined Method

The linear programming-refined method is the second placement strategy evaluated in this project. The 2D formulation is closely inspired by the LP-based floorplanning algorithm of Kim and Kim [3]. Their main idea is to separate the floorplanning problem into two parts: a discrete topology search over sequence pairs, and a continuous LP optimization for a fixed topology.

This project implements that idea for 2D floorplanning and then extends the same principle to 3D floorplanning using sequence triples. In this project, the extension to sequence triples, tier-aware constraints, TSV-aware costs, and thermal-aware penalties is part of our own experimental framework.

6.1 Method Overview

The LP-refined method is used after simulated annealing has identified a strong sequence representation. During the simulated annealing search, candidate topologies are evaluated using the construction method because it is fast. After the search terminates, the best sequence pair or sequence triple is fixed, and an LP model is built from the corresponding relative placement constraints.

The LP does not search over all possible topologies. Instead, it assumes that the relative ordering of blocks has already been determined. For 2D, this ordering comes from the final sequence pair. For 3D, it comes from the final sequence triple and the derived tier assignment. The LP then optimizes continuous placement variables such as block coordinates and soft-block dimensions.

This structure keeps the combinatorial search separate from the continuous optimization. The sequence representation handles discrete topology decisions, while the LP improves the final geometric realization of that topology.

6.2 LP Model

For a fixed sequence pair in 2D, the LP includes coordinate variables for every block:

$$x_i, \quad y_i \quad \forall i \in B.$$

For soft blocks, width and height are also variables:

$$w_i, \quad h_i.$$

For hard blocks, width and height are fixed according to the orientation chosen before LP refinement.

The chip outline is represented by:

$$W, \quad H.$$

If the experiment uses a fixed outline, these variables are fixed to the required outline:

$$W = W_0, \quad H = H_0.$$

Otherwise, they are continuous variables optimized by the LP.

For each net k , the LP introduces bounding-box variables:

$$L_k, \quad R_k, \quad B_k, \quad T_k,$$

representing the left, right, bottom, and top boundaries of the net bounding box. The net width and height are:

$$w_k^{net} = R_k - L_k,$$

$$h_k^{net} = T_k - B_k.$$

The center of block i is:

$$c_i^x = x_i + \frac{1}{2}w_i, \quad c_i^y = y_i + \frac{1}{2}h_i.$$

For every block i connected to net k , the LP enforces:

$$L_k \leq c_i^x \leq R_k,$$

$$B_k \leq c_i^y \leq T_k.$$

Therefore, minimizing $w_k^{net} + h_k^{net}$ minimizes the HPWL approximation for the net.

The LP objective is:

$$\min \quad \alpha(W + H) + \beta \sum_{k \in N} (w_k^{net} + h_k^{net}).$$

The term $W + H$ is used instead of WH because WH is nonlinear. This follows the linearized chip-size objective used in Kim and Kim [3]. In fixed-outline experiments, W and H are constants, so the chip-size term does not affect the LP solution directly.

The main geometric constraints are boundary and non-overlap constraints. Each block must lie inside the outline:

$$x_i \geq 0, \quad y_i \geq 0,$$

$$x_i + w_i \leq W,$$

$$y_i + h_i \leq H.$$

The non-overlap constraints are derived from the fixed sequence representation. If the sequence pair implies that block i is left of block j , the LP adds:

$$x_i + w_i \leq x_j.$$

If block i is below block j , it adds:

$$y_i + h_i \leq y_j.$$

Because the sequence representation fixes the relation for each block pair, these constraints are linear.

6.3 Coordinate and Dimension Refinement

The LP refinement improves the placement by solving for coordinates globally. In the construction method, coordinates are obtained through longest-path compaction. This produces a feasible compact placement for a fixed topology, but it does not globally optimize wirelength or soft-block shapes. The LP, in contrast, optimizes all coordinates and soft-block dimensions simultaneously.

For hard blocks, dimensions are fixed before LP refinement. In the 2D implementation, hard-block orientations are chosen by the construction method, following the structure of Kim and Kim [3]. The LP then treats those hard-block dimensions as constants.

For soft blocks, width and height remain variables subject to aspect-ratio constraints:

$$r_i^{\min} w_i \leq h_i \leq r_i^{\max} w_i.$$

The exact area constraint

$$w_i h_i \geq A_i$$

is nonlinear. To keep the model linear, the implementation uses a linear surrogate:

$$w_i + \alpha_i h_i \geq 2\sqrt{A_i \alpha_i}.$$

The parameter α_i is updated iteratively. After each LP solve, the implementation checks whether:

$$w_i h_i \geq A_i.$$

If the area constraint is violated, a new value of α_i is added and the LP is solved again. This iterative correction keeps the model linear while encouraging the soft-block dimensions to satisfy their required areas.

The LP-refined solution is accepted only if the solver reports feasibility and the soft-block area checks pass. Otherwise, the method keeps the construction solution or marks the LP refinement as infeasible.

6.4 Extension to 3D

The 3D LP-refined method extends the same fixed-topology idea to sequence triples. After simulated annealing finishes, the best sequence triple is fixed. The sequence triple determines which block pairs are separated horizontally, vertically, or by tier. The LP then refines the in-tier coordinates and soft-block dimensions.

For blocks on the same tier, the LP uses the same non-overlap constraints as in 2D. If block i is left of block j on the same tier:

$$x_i + w_i \leq x_j.$$

If block i is below block j on the same tier:

$$y_i + h_i \leq y_j.$$

For blocks assigned to different tiers, no in-plane non-overlap constraint is required, because they occupy different physical tiers.

The 3D LP therefore optimizes the continuous layout for a fixed sequence triple and fixed tier assignment. TSV count and thermal-aware penalties are evaluated as part of the 3D objective, but the current implementation does not solve a full MILP for TSV placement or tier assignment. Those decisions are encoded by the sequence triple and simulated annealing. A full mathematical formulation with binary tier, orientation, and TSV variables is left as future work.

Chapter 7

Framework Implementation

The implementation was designed as a modular C++17 framework for experimentation with 2D and 3D sequence-based floorplanning. The goal was not only to reproduce the main algorithmic structure of Kim and Kim’s 2D LP-refined method, but also to provide an extendable baseline for studying 3D sequence triple floorplanning. The implementation developed for this project is available in the project GitHub repository [16]. A separate companion repository implements a related 2D and 3D sequence-based floorplanning framework with a modular constraint system, including soft objective terms and hard feasibility constraints for TSV count and critical wirelength [19].

7.1 System Architecture

The framework is organized around four main stages: input parsing, topology search, placement and refinement, and output generation with visualization.

The input parser reads benchmark data and builds an internal floorplanning problem. The topology search stage uses simulated annealing over sequence pairs or sequence triples. The placement stage either constructs coordinates directly or applies LP refinement to the best topology. Finally, the output stage writes placement files, summary metrics, LP models, and visualization figures.

The same architecture is used for both 2D and 3D modes. The 2D modes operate on sequence pairs, while the 3D modes operate on sequence triples and tier-aware objective terms.

7.2 Core Modules

The main implementation modules are:

- **Data model:** stores blocks, nets, floorplanning parameters, and final solutions. Blocks may be hard or soft, and solutions store coordinates, dimensions, chip size,

wirelength, feasibility status, and objective value.

- **Sequence pair:** implements the 2D sequence-pair representation, validation, mutation, and relation decoding.
- **Sequence triple:** implements the 3D sequence-triple representation, mutation, tier-relation decoding, and layer/tier-aware topology interpretation.
- **Placement and construction:** builds horizontal and vertical precedence graphs, computes longest paths, assigns compact coordinates, and evaluates construction-based solutions.
- **LP floorplanner:** builds the LP model for a fixed sequence representation, including coordinate variables, soft-block dimensions, boundary constraints, net bounding boxes, and sequence-derived non-overlap constraints.
- **Solver interface:** isolates solver-specific code behind a common interface. HiGHS is used as the main open-source LP solver backend, while MOSEK support is included for comparison when available.
- **Annealer:** implements simulated annealing, neighborhood moves, temperature cooling, objective evaluation, and acceptance decisions.
- **Input/output:** writes placement CSV files, JSON summaries, exported LP or MPS models, and benchmark metadata.

This modular structure makes it possible to compare construction-only and LP-refined modes while sharing the same input representation and evaluation metrics.

7.3 Benchmark Parsing and Output Generation

The framework supports MCNC benchmark instances such as `apte`, `xerox`, `hp`, `ami33`, and `ami49`. The parser converts benchmark block and net descriptions into the internal problem representation. Each block is assigned an identifier, dimensions, type information, and optional aspect-ratio data. Nets are stored as lists of connected block identifiers.

After each run, the program writes a placement CSV file containing block names, coordinates, dimensions, type, and tier. It also writes a JSON summary containing the main experimental metrics:

objective, W , H , area, wirelength, TSV count, runtime.

The summary also stores the mode, solver, number of iterations, seed, number of blocks, number of nets, and feasibility status. This makes the output suitable for automated comparison across benchmark instances and algorithmic modes.

7.4 Visualization and Solver Integration

The project includes visualization scripts for both 2D and 3D results. The 2D visualization draws rectangular block placements and can also display 3D results as separate tier-by-tier 2D plots. A separate 3D visualization script produces a stacked view of the tiered placement. These figures are used to inspect whether the resulting floorplans are geometrically reasonable and to compare construction-based and LP-refined layouts.

The LP solver interface supports exporting models in standard LP or MPS format. This allows the same generated model to be inspected or solved externally using tools such as HiGHS or MOSEK. HiGHS is used as the default open-source solver in the experiments. MOSEK integration is used for solver comparison when the local MOSEK installation is available.

This solver abstraction allows for future work extending the same interface to additional commercial solvers or to MILP backends for explicit tier-assignment and TSV placement models.

Chapter 8

Experimental Setup

8.1 Benchmarks

The experiments were conducted on MCNC benchmark circuits, which are commonly used in floorplanning studies. The selected benchmarks were:

`apte`, `xerox`, `hp`, `ami33`, `ami49`.

These benchmarks provide hard-block module dimensions and netlist connectivity. The smaller benchmarks, such as `apte`, `xerox`, and `hp`, are useful for checking correctness and comparing methods quickly, while `ami33` and `ami49` provide larger test cases where runtime and optimization quality become more visible.

For the 2D experiments, each benchmark was placed on a single tier. For the 3D experiments, the same benchmark blocks and nets were distributed across three tiers. The 3D experiments therefore test whether sequence-triple floorplanning can reduce horizontal wirelength by allowing connected blocks to be stacked across tiers.

8.2 Compared Methods

Four main methods were compared:

1. **2D SA-CT**: simulated annealing over sequence pairs, with each candidate evaluated using the construction-based method.
2. **2D SA-CT-LP**: simulated annealing over sequence pairs using construction-based evaluation, followed by LP refinement on the best final sequence pair.
3. **3D SA-CT**: simulated annealing over sequence triples, with each candidate evaluated using the 3D construction-based method.

4. **3D SA-CT-LP**: simulated annealing over sequence triples using construction-based evaluation, followed by LP refinement on the best final sequence triple.

The comparison is designed to isolate two effects. First, comparing SA-CT with SA-CT-LP shows the effect of LP refinement. Second, comparing the 2D and 3D versions shows the effect of extending the representation from sequence pairs to sequence triples.

8.3 Evaluation Metrics and Parameters

The methods were evaluated using objective value, HPWL, TSV count, feasibility, and runtime. The objective value is a weighted cost combining wirelength, outline-related terms, and, in 3D, TSV and thermal-aware penalties. HPWL is used as the main wirelength estimate. TSV count is reported only for 3D experiments.

Runtime was measured as the total execution time of each run, including simulated annealing, construction-based evaluations, and final LP refinement when applicable. The same random seed was used across comparable runs to make the comparison more stable. Although the framework supports both HiGHS and MOSEK, the final benchmark matrix is reported using HiGHS. Preliminary solver comparisons showed negligible differences in the resulting objective values for the same exported LP models. Since the purpose of the experiments is to compare floorplanning methods rather than LP solver performance, and since rerunning all benchmarks with both solvers would be computationally expensive, HiGHS was selected as the primary solver for the reported results as an open-source solver.

The 3D objective includes TSV-related and thermal-aware terms. These terms are lightweight proxies rather than full physical simulation models. Their purpose is to penalize excessive vertical communication and concentrated stacking while keeping the experimental framework simple enough for comparison.

Chapter 9

Results and Discussion

9.1 2D Results

Table 9.1 summarizes the 2D results. The construction-only method often produced placements that violated fixed-outline or aspect-ratio constraints. This does not mean the construction method is useless; it is still effective as a fast evaluation method during simulated annealing. However, it shows that construction alone is less reliable as a final legalization step under fixed-outline constraints.

The LP-refined method produced feasible final placements for all reported 2D benchmarks. This supports the main motivation of the Kim and Kim-inspired approach: once a good sequence-pair topology has been found, LP refinement can improve the final coordinate realization and enforce feasibility more reliably.

Table 9.1: 2D results on MCNC benchmarks

Benchmark	Method	Objective	HPWL	Runtime (min)	Status
apte	SA-CT	1,930,103	740,904	0.2	outline/aspect violation
apte	SA-CT-LP	736,403	718,195	0.2	LP optimal
xerox	SA-CT	694,207	607,515	0.3	outline/aspect violation
xerox	SA-CT-LP	598,922	586,606	0.3	LP optimal
hp	SA-CT	886,113	268,142	0.3	outline/aspect violation
hp	SA-CT-LP	222,078	212,962	0.3	LP optimal
ami33	SA-CT	274,632	82,102	15.0	outline/aspect violation
ami33	SA-CT-LP	83,212	80,681	15.5	LP optimal
ami49	SA-CT	2,886,779	997,836	92.2	outline/aspect violation
ami49	SA-CT-LP	991,512	978,503	91.9	LP optimal

9.2 3D Results

Table 9.2 summarizes the 3D results. Unlike the 2D construction-only runs, the 3D construction method produced feasible placements for all reported benchmarks. The

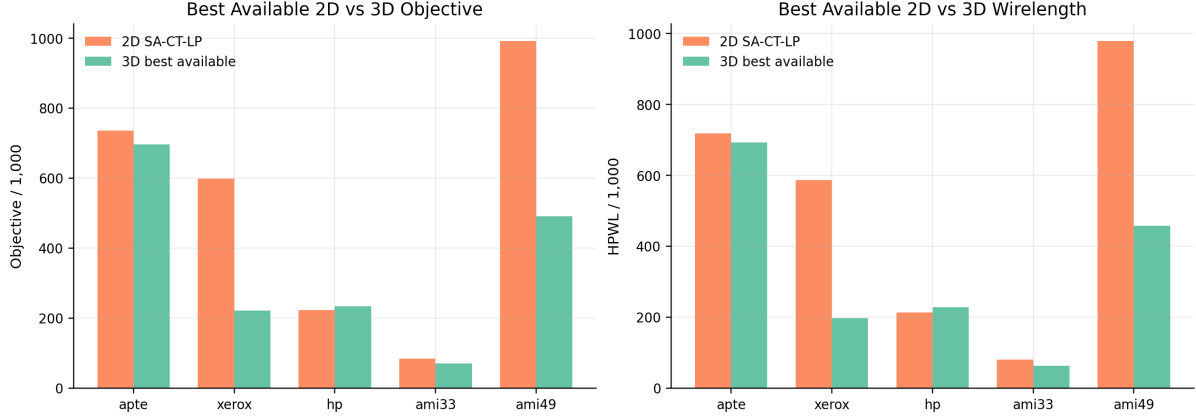


Figure 9.1: Comparison of 2D and 3D floorplanning results. The 3D sequence-triple formulation reduces wirelength on larger benchmarks by allowing connected blocks to be placed across multiple tiers.

LP-refined version further improved objective value and HPWL in every case.

The improvement is especially visible for larger benchmarks. For **ami49**, the 3D LP-refined method reduced HPWL from 487,319 to 458,056 compared with the 3D construction result. This shows that even after the sequence triple determines tier relations, there remains useful continuous freedom that LP refinement can exploit.

Table 9.2: 3D results on MCNC benchmarks

Benchmark	Method	Objective	HPWL	TSVs	Runtime (min)	Status
apte	SA-3D-CT	758,689	755,473	27	0.5	construction feasible
apte	SA-3D-CT-LP	696,137	692,970	27	0.5	3D LP optimal
xerox	SA-3D-CT	255,308	231,505	193	1.2	construction feasible
xerox	SA-3D-CT-LP	220,896	196,811	193	1.3	3D LP optimal
hp	SA-3D-CT	252,976	248,663	40	0.7	construction feasible
hp	SA-3D-CT-LP	233,003	228,431	40	0.7	3D LP optimal
ami33	SA-3D-CT	73,286	66,090	61	25.1	construction feasible
ami33	SA-3D-CT-LP	70,081	62,925	61	28.1	3D LP optimal
ami49	SA-3D-CT	520,329	487,319	273	206.0	construction feasible
ami49	SA-3D-CT-LP	491,302	458,056	273	218.6	3D LP optimal

9.3 Construction-Based vs LP-Refined Comparison

The main trend is that LP refinement improves solution quality after construction-based topology search. In 2D, the most important effect is feasibility: LP refinement produced valid fixed-outline placements where construction-only results often violated outline or aspect-ratio constraints.

In 3D, construction already produced feasible placements, but LP refinement improved the objective and wirelength. Table 9.3 shows the percentage objective improvement from 3D construction to 3D LP refinement.

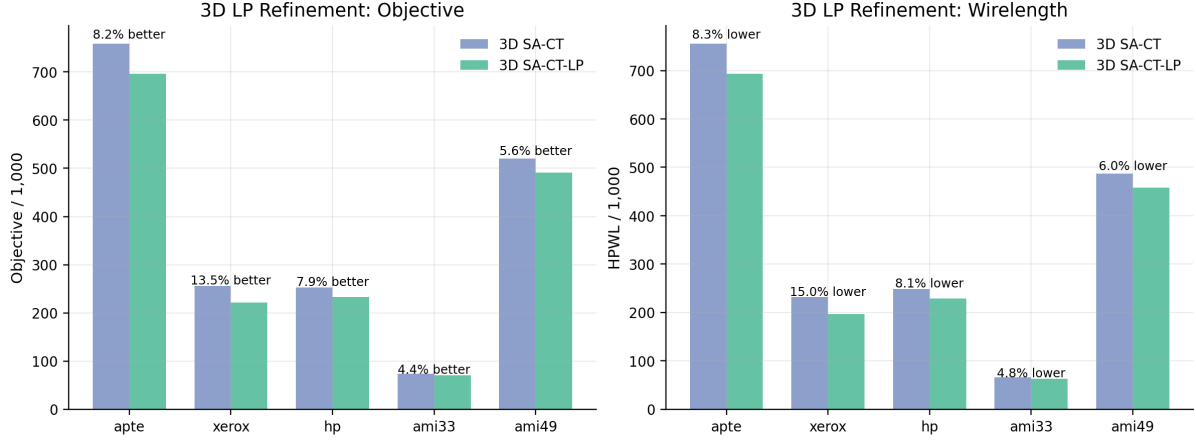


Figure 9.2: Effect of LP refinement on 3D floorplanning results. For each benchmark, SA-3D-CT is compared with SA-3D-CT-LP. LP refinement improves both objective value and HPWL after the sequence-triple topology is fixed.

Table 9.3: Objective improvement from 3D LP refinement

Benchmark	Objective Improvement
apte	8.2%
xerox	13.5%
hp	7.9%
ami33	4.4%
ami49	5.6%

These results support the hybrid design of the algorithm. Simulated annealing and construction are useful for exploring the discrete topology space efficiently, while LP refinement improves the final continuous placement for the best topology found.

9.4 Runtime and Trade-off Analysis

The runtime results show that 3D floorplanning is more expensive than 2D floorplanning. This is expected because the sequence-triple search space is larger and each candidate evaluation includes tier assignment, tier-aware compaction, TSV cost, and thermal-aware cost terms.

For small benchmarks, all methods finish quickly. For larger benchmarks, the runtime difference becomes significant. On **ami49**, the 2D LP-refined run took about 91.9 minutes, while the 3D LP-refined run took about 218.6 minutes.

The LP refinement itself is not the dominant source of runtime. For instance, on **ami49**, 3D SA-CT required approximately 206.0 minutes, while 3D SA-CT-LP required 218.6 minutes. This indicates that most of the computational cost arises from simulated annealing and repeated construction-based evaluations, rather than from the final LP

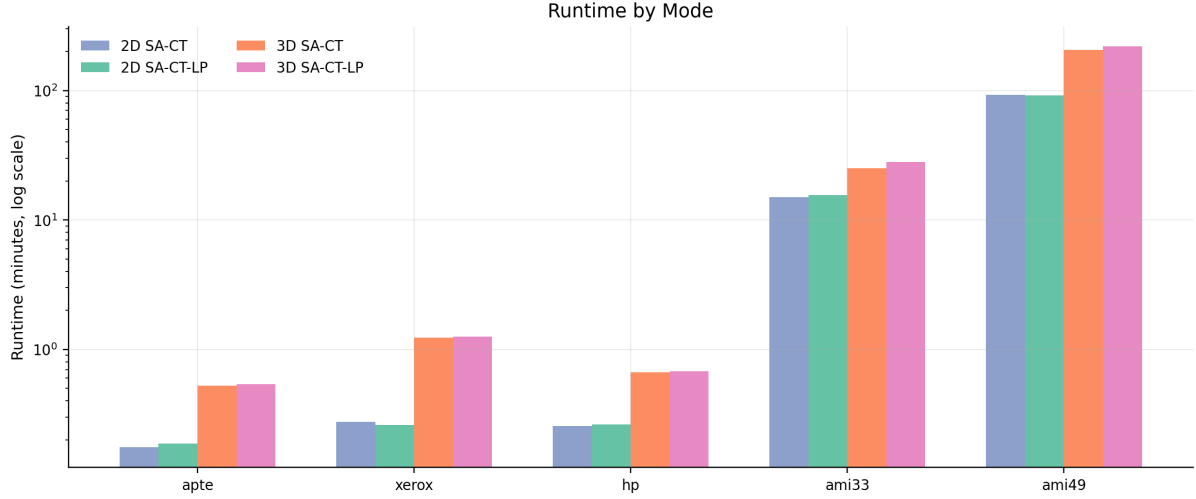


Figure 9.3: Runtime comparison for the larger MCNC benchmarks. The 3D methods require more runtime because simulated annealing evaluates a larger sequence-triple search space and includes tier-aware costs.

solve. Overall, the 3D approach offers improved wirelength and objective values, but these gains come with increased search cost and additional TSV overhead.

The results show that 3D floorplanning can significantly reduce wirelength, especially on larger benchmarks, but it requires more computation and introduces vertical interconnect costs.

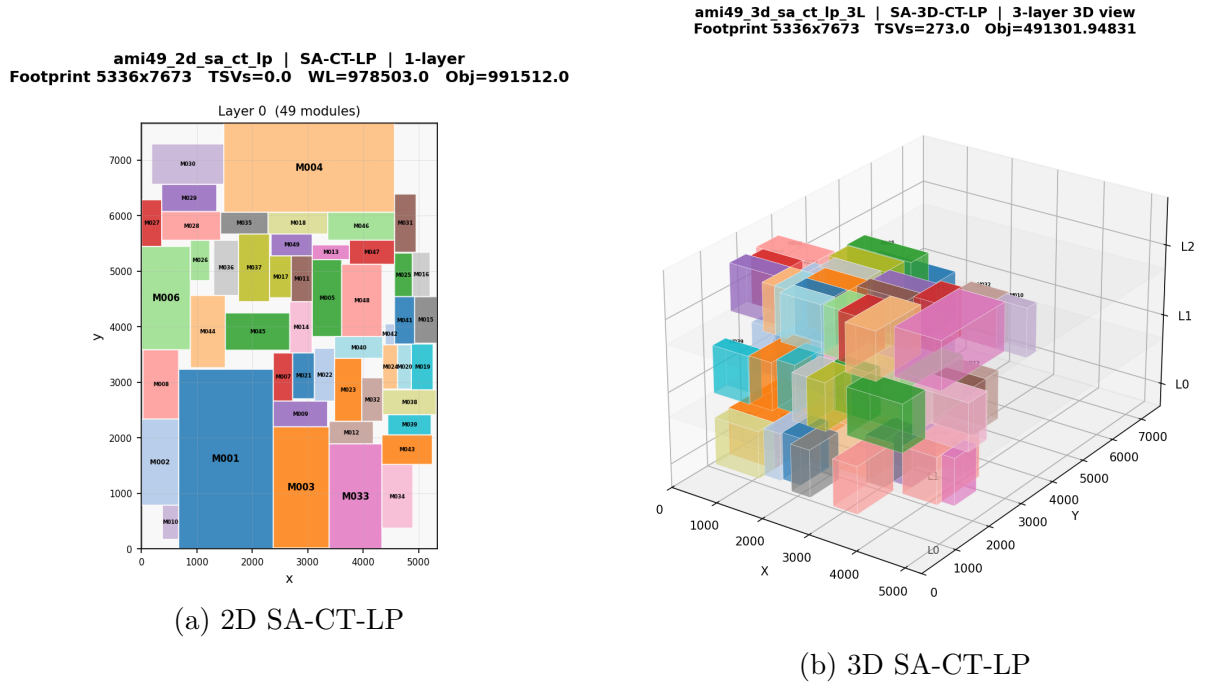


Figure 9.4: Qualitative visualization of the **ami49** LP-refined floorplans. The 2D floorplan places all blocks on one tier, while the 3D floorplan distributes blocks across three tiers, allowing vertical stacking and shorter horizontal connections.

Chapter 10

Conclusion and Future Work

10.1 Conclusion

This project implemented and compared sequence-based floorplanning methods for 2D and 3D layouts. The 2D method follows the structure of Kim and Kim’s sequence-pair and LP-refined floorplanning approach, while the 3D method extends the framework using sequence triples, tier-aware placement, TSV cost, and thermal-aware penalties.

The results show that simulated annealing with construction-based evaluation is useful for exploring floorplan topologies efficiently. LP refinement improves the final coordinate realization after the best topology has been found. In 2D, LP refinement was especially important for producing feasible fixed-outline placements. In 3D, LP refinement improved objective value and HPWL across all reported benchmarks.

The 3D extension demonstrated that sequence triples can reduce horizontal wirelength by allowing blocks to be distributed across multiple tiers. This benefit was strongest for larger benchmarks such as `ami33` and `ami49`. However, 3D floorplanning also increased runtime and introduced TSV-related overhead. Overall, the results show a clear quality-runtime trade-off.

10.2 Future Work

Future work should extend the 3D LP formulation into a full mixed-integer linear programming model. This would allow tier assignment, hard-block orientation, and TSV placement to be optimized explicitly rather than fixed by the sequence triple and construction stage.

The thermal model should also be improved. The current implementation uses a lightweight thermal-aware proxy, but a more realistic thermal model would allow stronger conclusions about hotspot reduction and thermal feasibility. Future extensions could also include pin-aware wirelength, routing congestion estimation, TSV keepout legalization,

and multi-seed experiments.

Future extensions could also investigate timing- or clock-aware floorplanning proxies, although detailed clock-tree synthesis lies beyond the scope of the current block-level framework.

Finally, the framework should be evaluated on larger and more diverse benchmark suites. This would help determine when 3D stacking consistently improves floorplan quality and when TSV or thermal costs outweigh the wirelength benefit.

Acknowledgment

The authors thank Dr. Hayk Grigoryan, their capstone supervisor, for his guidance, feedback, and support throughout the development of this project.

Bibliography

- [1] R. B. Singh, A. S. Baghel, and A. Agarwal, “A review on VLSI floorplanning optimization using metaheuristic algorithms,” in *2016 International Conference on Electrical, Electronics, and Optimization Techniques (ICEEOT)*, Chennai, India, 2016, pp. 4198–4202.
- [2] Hiroshi Murata, Kunihiro Fujiyoshi, Shigetoshi Nakatake, and Yoji Kajitani. Rectanglepacking-based module placement. In *The Best of ICCAD*, pages 535–548. Springer US, 2003.
- [3] J.-G. Kim and Y.-D. Kim, “A linear programming-based algorithm for floorplanning in VLSI design,” *IEEE Transactions on Computer-Aided Design of Integrated Circuits and Systems*, vol. 22, no. 5, pp. 584–592, 2003.
- [4] Lalin L Laudis, N. Ramadass, Shilpa Shyam, R. Benschwartz, V. Suresh. An Adaptive Symbiosis based Metaheuristics for Combinatorial Optimization in VLSI. *Procedia Computer Science*, Volume 167, 2020, Pages 205-212. ISSN 1877-0509, <https://doi.org/10.1016/j.procs.2020.03.197>.
- [5] Jieliang Lu, Qin Wang, Jing Xie, Zhigang Mao. (2013). TSVs-aware floorplanning for 3D integrated circuit. 2013 IEEE 10th International Conference on ASIC, 1–4. <https://doi.org/10.1109/asicon.2013.6812068>
- [6] Healy, M., Vittes, M., Ekpanyapong, M., Ballapuram, C. S., Lim, S. K., Lee, H.-H. S., Loh, G. H. (2007). Multiobjective Microarchitectural Floorplanning for 2-D and 3-D ICs. *IEEE Transactions on Computer-Aided Design of Integrated Circuits and Systems*, 26(1), 38–52. <https://doi.org/10.1109/tcad.2006.883925>
- [7] Srinivasan, B., Venkatesan, R. (2021). Multi-objective optimization for energy and heat-aware VLSI floorplanning using enhanced firefly optimization. *Soft Computing*, 25(5), 4159–4174. <https://doi.org/10.1007/s00500-021-05591-x>
- [8] Abdullah, D. M., Abdullah, W. M., Babu, N. M., Bhuiyan, Md. M. I., Nabi, K. M., Rahman, M. S. (2013). VLSI floorplanning design using clonal selection algorithm. 2013 International Conference on Informatics, Electronics and Vision (ICIEV), 1–6. <https://doi.org/10.1109/iciev.2013.6572664>

- [9] Tang, M. (2008). A New Greedy Algorithm for VLSI Floorplan Optimization. 2008 International Conference on Computer Science and Software Engineering, 1126–1129. <https://doi.org/10.1109/csse.2008.1560>
- [10] Huang, Y., Zhou, Q., Cai, Y., Yan, H. (2009). A thermal-driven force-directed floorplanning algorithm for 3D ICs. 2009 11th IEEE International Conference on Computer-Aided Design and Computer Graphics, 497–502. <https://doi.org/10.1109/cadcg.2009.5246852>
- [11] Linfu Xiao, Sinha, S., Jingyu Xu, Young, E. F. Y. (2010). Fixed-outline thermal-aware 3D floorplanning. 2010 15th Asia and South Pacific Design Automation Conference (ASP-DAC), 561–567. <https://doi.org/10.1109/aspdac.2010.5419822>
- [12] Tsai, M.-C., Wang, T.-C., Hwang, T. (2011). Through-Silicon Via Planning in 3-D Floorplanning. IEEE Transactions on Very Large Scale Integration (VLSI) Systems, 19(8), 1448–1457. <https://doi.org/10.1109/tvlsi.2010.2050012>
- [13] Li, C.-R., Mak, W.-K., Wang, T.-C. (2013). Fast Fixed-Outline 3-D IC Floorplanning With TSV Co-Placement. IEEE Transactions on Very Large Scale Integration (VLSI) Systems, 21(3), 523–532. <https://doi.org/10.1109/tvlsi.2012.2190537>
- [14] Li, Z., Hong, X., Zhou, Q., Zeng, S., Bian, J., Yu, W., Yang, H. H., Pitchumani, V., Cheng, C.-K. (2007). Efficient Thermal via Planning Approach and Its Application in 3-D Floorplanning. IEEE Transactions on Computer-Aided Design of Integrated Circuits and Systems, 26(4), 645–658. <https://doi.org/10.1109/tcad.2006.885831>
- [15] Singh, R. B., Baghel, A. S. (2017). Dead space reduction of floorplan using simulated annealing algorithm. 2017 International Conference on Energy, Communication, Data Analytics and Soft Computing (ICECDS), 2108–2112. <https://doi.org/10.1109/ICECDS.2017.8389822>
- [16] Z. Grigoryan and A. Ishkhanyan, “2D and 3D Floorplanning,” GitHub repository, 2026. [Online]. Available: <https://github.com/zgrigoryan/2d-and-3d-floorplanning>
- [17] C. Li, Y. Yin, C. Xue, Z. Wang, F. Bai, Y. Guo, X. Jiang, Q. Wu, Y. Xie, and G. Sun, “Hardware-Software Co-design for 3D-DRAM-based LLM Serving Accelerator,” *arXiv preprint arXiv:2603.04797*, 2026. doi: 10.48550/arXiv.2603.04797.
- [18] A. Teman, “Lecture 8: Clock Tree Synthesis,” *Digital VLSI Design*, Bar-Ilan University, Semester A 2016–2017, Jan. 22, 2017. [Online]. Available: <https://www.eng.biu.ac.il/temanad/files/2017/02/Lecture-8-CTS.pdf>

- [19] Z. Grigoryan and A. Ishkhanyan, “2D and 3D Constraint-Based Greedy Floorplanning,” GitHub repository, 2026. [Online]. Available: <https://github.com/anna-ishkhanyan/2D-and-3D-Floorplanning-Simulated-Annealing-and-Linear-Programming-Approaches>